Software Architecture and Design Specification (SAD)

Hospital Management System

Project	Hospital Management System
Version	1.0
Authors	Project Team
Date	14-11-2026
Status	Updated working copy
Related Documents	SRS v1.0; Software Test Plan v1.0

Revision History

Version	Date	Author	Change Summary
1.0	14-11-2026	Project Team	Architecture and design specification aligned to supplied repository implementation and SAD template.

Approvals

Role	Name	Signature / Date
Project Guide / Coordinator	_____	_____
Team Representative	_____	_____

1. Introduction

1.1 Purpose

This document describes the implemented architecture and design of the Hospital Management System, including frontend/backend boundaries, major components, data handling, API contracts, security controls and representative workflows.

1.2 Scope

The specification covers the React/Vite browser client, Node.js/Express API, authentication, appointments, clinical records, prescriptions, report files, medicine orders/demo payments and bed management.

1.3 Audience

Developers, QA engineers, project evaluators, maintainers and security reviewers.

1.4 Definitions

Term	Meaning
SAD	Software Architecture and Design
REST API	HTTP-based application programming interface
JWT	JSON Web Token
OTP	One-Time Password
RBAC	Role-Based Access Control
Multer	Express middleware used for multipart file uploads

2. Document Overview

2.1 How to Use This Document

Sections 3 and 4 describe the architecture and detailed design. API tables list the observed backend routes. The sequence flows describe the request path for core use cases. Security and operational limitations are called out where the repository does not implement a production-grade capability.

2.2 Related Documents

- Hospital_Management_System_SRS.docx
- Hospital_Management_System_Test_Plan.docx
- Hospital_Management_System_Use_Case_Flow.pdf
- Hospital_Management_System_Use_Case_Diagram_Clean.drawio

3. Architecture

3.1 Goals and Constraints

- Separate browser presentation from server-side validation and business rules.
- Provide role- and ownership-aware access to patient and doctor operations.
- Support appointment, clinical record, report, medicine and bed workflows through API endpoints.
- Keep the architecture understandable for a small academic project.
- Constraint: the supplied backend uses in-memory arrays for core entities and does not configure a persistent database.
- Constraint: OTP is logged to the development console; no SMS provider is integrated.
- Constraint: payment is a demonstration endpoint; no production gateway is configured.

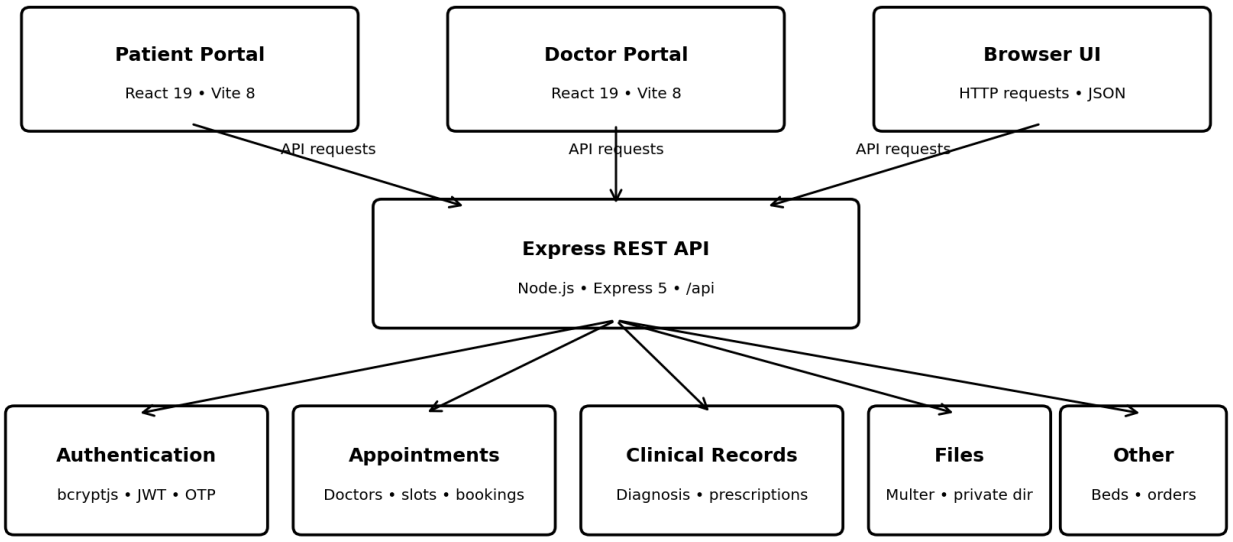
3.2 Stakeholders and Concerns

Stakeholder	Primary Concerns
Patients	Simple registration, secure login, appointments, access to own records and prescriptions.
Doctors	Assigned appointments, relevant patient history, diagnosis and prescription workflows.
Hospital operations	Bed visibility and reliable service availability.
Developers	Modular frontend/backend separation, maintainable API contracts and clear validation.
QA / Evaluators	Traceable requirements, testable endpoints, clear limits of the demonstration implementation.

3.3 Component Diagram

The following view reflects the code in the supplied ZIP. The older SAD.jpeg in the repository depicts a persistent Hospital Database, but the supplied server.js uses in-memory arrays and does not establish a database connection. The updated diagram therefore records the implementation actually present.

Hospital Management System – Implemented Component View



Data in the supplied backend is primarily held in in-memory arrays; report attachments are stored on disk. No persistent SQL/NoSQL database connection is configured in this repository.

3.4 Component Descriptions

Component	Technology / Responsibility
Patient Portal	React 19/Vite UI for registration, OTP, login, appointments, medical history, bills, prescriptions and medicine purchase.
Doctor Portal	React UI flows for doctor authentication, assigned appointments, diagnosis, reports, prescriptions and bed management.
Express API	Node.js/Express 5 server; validates requests, authenticates tokens, applies business rules and returns JSON responses.
Authentication	bcryptjs password hashing, JWT session tokens, SHA-256 OTP hash and attempt/expiry checks.
Appointment Module	Doctor directory, slot availability, booking, patient appointment listing and cancellation.
Clinical Records Module	Patient medical history, doctor diagnosis and prescription data.
File Management	Multer upload handling; private report directory; authorized report download route.
Medicine / Payment Module	Doctor medicine catalogue, patient prescribed medicine listing, order creation and demo payment.
Bed Management	Doctor-authorized bed listing, ward filtering, allocation and release.
Data Storage	Core users, appointments, records, prescriptions, beds and orders use in-memory arrays. Report attachments are written to local disk.

3.5 Chosen Architecture Pattern and Rationale

The implementation follows a simple client-server, layered web architecture. The React presentation layer communicates with the Express API layer over HTTP. The API layer contains route handlers, authentication middleware, validation and business logic. Data is currently held in process memory, with uploaded report files stored on the server filesystem. This is an implementation description rather than a claim of a fully separated microservice or persistent database architecture.

3.6 Technology Stack and Data Stores

Layer	Technology / Store	Observed Use
Frontend	React 19, React DOM, Vite 8	Single-page browser application.
Backend	Node.js, Express 5	HTTP API and business logic.
Authentication	bcryptjs, jsonwebtoken	Password hashing and signed eight-hour JWTs.
Configuration	dotenv	Loads JWT_SECRET and doctor account variables from backend/.env.
File upload	Multer 2.x	PDF/JPEG/PNG upload, max 5 MB.
Core records	In-memory JavaScript arrays	Users, doctors, appointments, beds, diagnoses, prescriptions, reports metadata and orders.
Report files	Local private-reports directory	Uploaded/demo report attachments; access mediated through API.

3.7 Risks and Mitigations

Risk	Impact	Mitigation / Next Step
In-memory data loss on restart	User, appointment, prescription, bed and order data are not durable.	Introduce a persistent database and migration/backup plan.
Development OTP logged to console	OTP exposure if deployed without changing delivery approach.	Integrate an SMS provider; remove OTP logging; retain expiry and attempt limits.
Demo payment flow	Does not represent actual payment authorization/settlement.	Integrate a trusted payment gateway and verify server-side payment callbacks.
No TLS configuration in local project	Credentials/tokens could be exposed on an untrusted network.	Deploy behind HTTPS with secure headers and environment-specific CORS.
Local report file storage	Disk loss, scaling and backup concerns.	Use managed private object storage, access policies and backup/retention controls.
No persistent audit trail evidenced	Sensitive record access/change events may not be independently traceable.	Add immutable or access-controlled audit events.

3.8 Traceability to Requirements

Requirement Group	Architecture Component
Registration, OTP, login	Authentication Service / Express API
Doctor listing, availability, booking, cancellation	Appointment Module
Medical history and diagnosis	Clinical Records Module
Prescription creation and retrieval	Prescription Module
Reports upload/download	File Management Module
Medicine ordering and demo payment	Medicine / Payment Module
Bed availability, allocation and release	Bed Management Module
Security, access control and validation	Authentication middleware and route-level checks

3.9 Security Architecture

- Passwords are hashed using bcryptjs before storage; login compares the submitted password with the stored hash.
- Patient and doctor tokens are verified using JWT middleware; doctor routes check the doctor role.
- Patient endpoints use authenticated patient identity to scope records and appointment access.
- Medical file downloads allow only the report's patient or its assigned doctor.
- OTP values are stored as SHA-256 hashes, expire after five minutes and are blocked after five incorrect attempts.
- Upload middleware permits PDF, JPEG and PNG files up to 5 MB.
- JWT_SECRET and doctor credentials are configured through environment variables.

Security gaps for production: no configured TLS termination, no external SMS delivery, no production payment gateway, no persistent audit store, and no durable database are evidenced in the supplied ZIP.

4. Design

4.1 Design Overview

The frontend calls the backend API using fetch. Authenticated requests pass a bearer token. The backend validates the request, checks identity/role/ownership, applies the workflow rules, updates in-memory records or writes an attachment, and returns a JSON response.

4.2 UML Sequence / Interaction Flows

4.2.1 Patient Registration and OTP Verification

Step	Actor / Component	Interaction
1	Patient → React UI	Enter name, email, phone and password; submit registration.
2	React UI → POST /api/auth/register	Send registration details as JSON.
3	Express API	Validate fields and uniqueness; hash password; generate OTP; store OTP hash and expiry.
4	Express API → Development console	Print development OTP (no SMS provider is configured).
5	Patient → React UI	Enter the six-digit OTP.
6	React UI → POST /api/auth/verify-otp	Send phone and OTP.
7	Express API	Validate format, expiry and attempt count; compare hashes; mark phone verified.
8	React UI	Display verification result; patient can proceed to email/password login.

4.2.2 Appointment Booking

Step	Actor / Component	Interaction
1	Patient → React UI	Select doctor and appointment date.
2	React UI → GET /api/doctors	Retrieve configured doctors.
3	React UI → GET /api/doctors/:doctorId/slots?date=...	Retrieve available time slots.
4	Patient → React UI	Select an available time and submit.
5	React UI → POST /api/appointments	Send doctorId, date and time with bearer token.
6	Express API	Authenticate patient; validate doctor/date/time; check slot availability.
7	Express API	Create appointment linked to authenticated patient and doctor.
8	React UI	Show booking confirmation and refresh patient's appointment list.

4.3 API Design

Base path: /api. Request/response bodies are JSON unless multipart upload is specified. Protected endpoints use Authorization: Bearer <JWT>.

Method	Endpoint	Access	Purpose
POST	/auth/register	Public	Register patient; returns verification-required response.
POST	/auth/verify-otp	Public	Verify phone OTP.
POST	/auth/login	Public	Authenticate verified patient; returns JWT.
POST	/auth/doctor-login	Public	Authenticate configured doctor; returns doctor JWT.
GET	/health	Public	Health status.
GET	/doctors	Public	List doctors.
GET	/doctors/:doctorId/slots	Public	List available slots for a date.
POST	/appointments	Patient	Create appointment.
GET	/appointments/me	Patient	List own appointments.
PATCH	/appointments/:id/cancel	Patient	Cancel own booked appointment.
GET	/beds	Doctor	List beds, optional ward filter and summary.
PATCH	/beds/:id/allocate	Doctor	Allocate available bed.
PATCH	/beds/:id/release	Doctor	Release occupied bed.
GET	/doctor/appointments	Doctor	List appointments assigned to doctor with related patient details.
POST	/doctor/appointments/:id/diagnoses	Doctor	Save diagnosis for assigned appointment.
GET	/medicines/catalogue	Doctor	List medicine catalogue.
POST	/doctor/appointments/:id/prescriptions	Doctor	Create validated prescription.
GET	/prescriptions/me	Patient	List own prescriptions.
POST	/doctor/appointments/:id/reports	Doctor	Create report metadata and optional multipart file upload.
GET	/patient/medical-records	Patient	List own medical records and report metadata.
GET	/reports/:id/download	Patient or assigned doctor	Download authorized report file.
GET	/medicines/prescribed	Patient	List medicines from own prescriptions.
POST	/medicine-orders	Patient	Create order for prescribed medicines.
POST	/medicine-orders/:id/pay-demo	Patient	Execute demonstration payment flow.
GET	/medicine-orders/me	Patient	List own medicine orders.
GET	/patient/bills/me	Patient	List own bills.
POST	/patient/bills/demo	Patient	Create demonstration bill.

4.4 Error Handling, Logging and Monitoring

The API returns HTTP status codes and JSON message fields for validation failures, authentication failures, missing resources, conflicts and authorization failures. Examples include 400 for invalid input, 401 for missing/invalid authentication, 403 for forbidden role/ownership, 404 for missing resources, 409 for conflicting state, 410 for expired OTP, and 429 for excessive OTP attempts. The backend logs development OTPs and selected diagnostic/error messages; sensitive production logging controls and centralized monitoring are not configured in the supplied ZIP.

4.5 UX Design

The interface is organized around patient and doctor workflows, with form validation, loading states, status messages, appointment selection, medical record views and medicine purchase interactions. The supplied screenshots document registration, OTP, appointments, bed availability, medical history, prescriptions, bills, payments and downloads.

4.6 Open Issues and Next Steps

- Replace in-memory arrays with a persistent relational or document database and add schema/index/migration definitions.
- Integrate a real OTP/SMS provider; remove development OTP console output.
- Integrate a production payment gateway; verify payment status server-side.

- Configure HTTPS, secure headers, CORS allowlists and production secret management.
- Add durable audit logging, backups, retention and monitoring.
- Define a separate staff role and staff-specific authorization only if the staff portal remains in scope.
- Implement and verify regional-language support if retained as an NFR.

5. Appendices

5.1 Glossary

JWT – JSON Web Token; OTP – One-Time Password; API – Application Programming Interface; RBAC – Role-Based Access Control; SRS – Software Requirements Specification; STP – Software Test Plan.

5.2 References

- Supplied Hospital Management System source ZIP (backend/server.js and frontend/src).
- Supplied SRS_SE_PROJECT.xlsx and Hospital_Management_System_Test_Plan.xlsx.
- Supplied Hospital_Management_System_Use_Case_Flow.pdf and UML diagram assets.

5.3 Tools

React, Vite, Node.js, Express, Visual Studio Code, draw.io and Microsoft Word.